Shreyash Neeraj

shreyashn.dev@gmail.com | +91-8858190988 | portfolio-tech-nu.vercel.app
linkedin.com/in/shreyashneeraj | github.com/shreyashsri79

Targeting a Backend / Systems Engineering role with a long-term focus on distributed systems design, building expertise in fault-tolerant architectures, protocol engineering, and scalable infrastructure to grow into a Systems Designer role.

Professional Summary

- Systems-focused engineer building in **Go, Python, Java, Kotlin, and SQL** across internships and independent projects in distributed systems, kernel I/O pipelines, and protocol engineering.
- Migrated eSim on Cloud's schematic editor from the unmaintained mxGraph engine to a custom React + SVG canvas, preserving save-format and API contracts of **25 downstream modules** via a facade adapter.
- Replaced mxGraph netlist generation with a coordinate-keyed **DSU connectivity compiler** with ERC checks, and shipped a KiCad schematic import pipeline using Prim-MST + Manhattan auto-routing, locked behind **60+ tests**.
- Diagnosed six structural bottlenecks in OABLT's parallel-TCP engine by reading the byte path alone (a mutex serialising concurrent `pwrite(2)`, a 4 KB `bufio` buffer, and split header/payload writes), lifting LAN throughput from **15 Mb/s to 148 Mb/s** (477 Mb/s on a direct hotspot).
- Built OpenAir, a spec-first QUIC device-continuity platform in Go: **one connection per peer** carrying files, clipboard, notifications, remote filesystem, screen mirror and input behind a versioned **8-byte envelope**, over pinned-key Ed25519 TLS 1.3 with no CA and no cloud in the data path.
- Built concurrent Node.js microservices with custom PostgreSQL and Redis connection pooling for ShopCardd, sustaining **2,000+ req/s** peak load; reduced CI-caught runtime errors by **40%** via Jest pipelines.

Education

Vellore Institute of Technology, Bhopal Campus
Bachelor of Technology in Computer Science (B.Tech CSE), Sep. 2024 - Oct. 2028

Technical Skills

Core Competencies: Distributed Systems Design, QUIC / TCP / UDP Protocol Engineering, P2P & Mesh Networking, Concurrent Systems (Go), Kernel I/O (`pwrite`/`writev`), Microservices Architecture, EDA Tooling & Canvas Engineering, Custom Wire Protocols, E2E Encrypted Networking, Lock-Free Programming

Languages: Go, Java, Kotlin, TypeScript, JavaScript, Python, SQL

Systems & Backend: QUIC (quic-go), TCP/UDP Sockets, mDNS/Zeroconf, Positional I/O (`pwrite`), Scatter/Gather I/O (`writev`), Goroutines/Channels, Django, Celery, Docker, React + SVG, DSU Algorithms

Frameworks & Tools: Gin, Jetpack Compose, React Native, Express.js, Next.js, Node.js, Git, Docker, Android Studio, Postman

Databases: PostgreSQL, Redis, MongoDB, MySQL

Work Experience

FOSSEE, IIT Bombay: eSim on Cloud, Mumbai, India
Project Intern (Summer Fellowship), Mentor: Sumanto Kar, May 2026 - July 2026

- Migrated eSim on Cloud's schematic editor from the unmaintained mxGraph library to a custom React + SVG canvas over an **immutable JSON document store**; preserved the legacy mxGraph XML save format and all API contracts of **25 downstream modules** via a facade and a bidirectional serialization adapter (toLegacyXml / fromLegacyXml), leaving the backend, viewer, and simulation pipeline untouched.
- Replaced mxGraph's DFS-over-terminals netlist generation with a **coordinate-keyed Disjoint Set Union (DSU)** connectivity compiler (path compression + union-by-rank); implemented a KiCad S-expression schematic import pipeline that extracts net groups and redraws wires via Prim-MST + obstacle-avoiding Manhattan router, validated against **all real gallery schematics** from the backend fixtures.
- Isolated high-frequency gesture state (wire draft, drag ghost, snap indicator) into a separate `interactionStore`, limiting per-mousemove re-renders to a **single overlay component** while hundreds of memoized canvas nodes bail out by identity; built a 47-test pure-logic suite (no DOM) covering geometry, DSU, welding, obstacle avoidance, rubber-banding, and full legacy-import round-trips.

- Shipped a server-side HDL simulation service for **Verilog, SystemVerilog, and VHDL** on Django and Celery async queues, packaging Icarus Verilog and NVC compiler toolchains via multi-stage Docker builds on an EOL Alpine base image; verified correctness with **22 end-to-end tests** against deployed containers.
- Led the production **PostgreSQL 13 to 18** upgrade: risk-analysed a 2020-era Django/psycopg2 client stack against SCRAM-SHA-256 auth and the official image's silent `PGDATA` relocation (which would destroy data on container recreation), then proved compatibility through a live three-layer verification harness (server, driver, ORM) and pinned the previously floating production image tag.

ShopCardd-Discount Ka JuGaad, Remote, India
Software Development Engineer (SDE) Intern, Sep. 2025 - Nov. 2025

- Built a concurrent microservices backend for an e-commerce platform in Node.js and Express with **custom PostgreSQL and Redis connection pooling** and memory-efficient stream processing, sustaining peak loads of **2,000+ requests per second**.
- Developed a cross-platform **React Native** mobile app for customer checkout and catalog browsing; optimized JSON payload serialization over the JS-to-Native bridge and introduced lock-free local state caching, cutting **UI thread latency by 35%**.
- Reduced critical post-production runtime errors by **40%** by architecting automated CI/CD pipelines with Jest integration and unit tests, achieving **85%+** code coverage.

Projects

OpenAir | *Go, QUIC (vendored quic-go + BBR), Ed25519, Protobuf, mDNS, STUN, Android/Kotlin* | GitHub, July 2026 - Present

- Rebuilt OABLT's transfer model into a cross-platform device-continuity platform (Linux, Windows, macOS, Android): files, clipboard, notifications, remote filesystem, screen mirror and remote input all ride a **single QUIC connection per peer**, multiplexed through a versioned **8-byte envelope** (`ver`, `capID`, `msgType`, `length`) over protobuf payloads; unknown capability and message IDs are ignored rather than fatal, so message types stay additive and **mixed-version fleets** interoperate.
- Implemented a **zero-trust identity layer**: Ed25519 device keys with TLS 1.3 pinned to raw public keys (no CA, no hostname check); pairing is TOFU-hardened by a 6-digit Short Authentication String (SAS = SHA-256(transcript) first 4 bytes, mod 1,000,000) with **no skip-verification path in the protocol**, and unattended (Owned) access needs a second privilege key sealed under Argon2id, unsealed only for a 6-hour per-device unlock in `mlock`ed memory.
- Wrote the connectivity substrate: a session starts on a self-hosted DERP-style relay in **about 1 RTT** while LAN dial and STUN-reflexive UDP hole-punch race in the background. A custom `net.PacketConn` keys each peer by DeviceID rather than `ip:port`, so promoting a punched path is a map write: QUIC above sees no address change, resets no streams and **restarts no in-flight transfer**.
- Ported OABLT's transfer engine onto a **vendored quic-go fork carrying BBRv1** and a Windows USO send path: **N parallel data streams** carrying 12-byte raw chunk frames (offset u64, size u32) written via `WriteAt()`, per-chunk SHA-256 manifests and resume from verified-chunk sets. Built range-read `remotefs` on top, publishing a loopback URL so any player's ordinary `Range` requests seek a multi-GB remote file without downloading it. Ran the project **spec-first**: a 1,255-line normative wire specification, a threat model and an **append-only decision log numbered to D-86** precede the code.

OABLT | *Go, Kotlin/Jetpack Compose, TCP Sockets, mDNS/NSD, pwrite/writev* | GitHub, April 2026 - May 2026

- Built a serverless LAN file transfer system (Go CLI and a Kotlin/Jetpack Compose Android app) on a **control/data split**: one TCP control connection carries a newline-JSON handshake and a bandwidth/RTT probe, then a worker pool opens **parallel persistent TCP connections**, each streaming chunks sized from the measured **bandwidth-delay product** (BDP divided by worker count, clamped to 256 KB to 4 MB) and framed by a 12-byte little-endian {`offset, size`} header that the receiver reassembles with positional `WriteAt`, so chunks may land in any order; end-to-end SHA-256 verification and zero-config mDNS/NSD service discovery throughout.
- Found **six structural bottlenecks by code reading alone, no profiler**: a mutex serialising POSIX-safe concurrent `pwrite(2)`, a 4 KB `bufio` buffer costing a syscall per header, two reflection-based `binary.Read` calls per 12-byte header, a `FileChannel` locked across workers, split header/payload writes and per-chunk allocation. Fixed them with 8 MB read buffers, per-worker file descriptors, `writev(2)` gather writes and reused buffers, taking throughput from **15 Mb/s to 148 Mb/s** on 5 GHz WiFi and **477 Mb/s** on a direct hotspot.
- Published a performance study of parallel-TCP scaling under `tc netem` (180 ms +/- 60 ms delay, 8% loss): throughput peaks at **about 192 Mb/s with 10 workers** and collapses to about 98 Mb/s at 50; optimal worker count tracks RTT and loss.

Certifications

Inclusive Open Source Community Orientation (LFC102) (Linux Foundation, 2026), Open Source Software (VITYarthi, 2026), Cloud Computing (NPTEL, IIT Kharagpur, Elite, 2025), Fundamentals of AI and ML (VITYarthi, 2025)